

Análise Comparativa de Paradigmas Algorítmicos na Quebra de Senhas: Um Estudo de Caso em Pentesting

Helian Silva

Maio de 2026

Resumo

Este artigo apresenta uma análise comparativa empírica e teórica de quatro paradigmas algorítmicos — Força Bruta, Divisão e Conquista, Programação Dinâmica e Algoritmos Genéticos — aplicados à quebra de senhas em cenários de Pentesting. Diante do desafio de explorar um espaço impraticável de 62^8 combinações, o estudo demonstra formalmente as reduções de complexidade de $O(k^n)$ para $O(2 \cdot k^{n/2})$ através da divisão do problema, e para $O(n \cdot k)$ no pior caso via memoização posicional. Além de avaliar o desempenho prático por meio do tempo de execução, chamadas ao Oráculo e consumo de memória, o trabalho discute a viabilidade dessas abordagens frente a mecanismos de defesa (como bloqueios de IP), a representação prática do custo de validação e a relação de custo-benefício (trade-off) entre espaço e tempo em sistemas reais de segurança da informação.

1 Introdução

No contexto da segurança da informação, a simulação de quebra de senhas é vital para avaliar a resiliência dos mecanismos de autenticação. Como a busca exaustiva por uma senha de 8 caracteres exige a exploração de um espaço de 62^8 combinações, impondo um custo computacional impraticável, torna-se essencial a aplicação de estratégias mais eficientes.

Este trabalho compara quatro paradigmas algorítmicos: Força Bruta, Divisão e Conquista, Programação Dinâmica e Algoritmos Genéticos. O documento demonstra formalmente como a complexidade teórica cai de $O(k^n)$ para $O(2 \cdot k^{n/2})$ através da divisão do problema, e como a utilização da memoização posicional reduz drasticamente as consultas ao sistema de validação (Oráculo) para $O(n \cdot k)$ no pior caso.

Além de apresentar a análise empírica do tempo de execução, consumo de memória e contagem de chamadas ao servidor, este relatório responde a questões práticas associadas ao tema. Serão abordadas a viabilidade desses ataques em cenários reais com bloqueio de IP, a representação prática do custo do oráculo (*Fitness Score*) e a relação de custo-benefício (*trade-off*) entre espaço e tempo exigida pelas otimizações.

2 Resultados Experimentais

Para garantir a confiabilidade e a estabilidade das métricas, foi realizada uma bateria de testes contendo 13 execuções independentes para cada um dos quatro algoritmos. O alvo consistiu em uma senha controlada de 8 caracteres alfanuméricos.

Abaixo estão os resultados consolidados, apresentando a média do Tempo Total de Execução, a quantidade exata de consultas ao sistema de validação (Hits) e a análise analítica do consumo de memória associado a cada abordagem.

2.1 Força Bruta (Busca Exaustiva)

- **Tempo Total de Execução:** 0,1909 segundos (média de 13 rodadas).
- **Contagem de *Hits*:** 14.784.025 consultas ao Oráculo.
- **Consumo de Memória:** $O(1)$. É a estratégia de menor impacto espacial. Exige apenas a alocação mínima para variáveis de controle de laço e para a manipulação da *string* temporária em escopo local, mantendo uma pegada de memória constante independente do tempo de execução.

2.2 Divisão e Conquista

- **Tempo Total de Execução:** 0,00018 segundos (média de 13 rodadas).
- **Contagem de *Hits*:** 7.691 consultas ao Oráculo.
- **Consumo de Memória:** $O(1)$. Extremamente baixo. O algoritmo armazena apenas as metades particionadas (duas *strings* parciais de 4 caracteres) na pilha de execução.

2.3 Programação Dinâmica (Memoização)

- **Tempo Total de Execução:** 0,000022 segundos (média de 13 rodadas).
- **Contagem de *Hits*:** 18 consultas ao Oráculo.
- **Consumo de Memória (Crítico):** A eficiência extrema desta estratégia custa espaço. O consumo teórico no pior caso é delimitado por $O(n \cdot k \cdot n)$ para armazenar prefixos e evitar o reprocessamento. No experimento controlado, a tabela de memoização e o dicionário de prefixos comuns ocuparam um espaço irrisório (dezenas de bytes). Contudo, em uma aplicação real escalonada, manter dicionários massivos de prefixos (como *Rainbow Tables*) na RAM exige uma quantidade substancial e crítica de memória.

2.4 Algoritmos Genéticos (Heurística)

- **Tempo Total de Execução:** 0,00076 segundos (média de 13 rodadas).
- **Contagem de *Hits*:** 1.810 consultas ao Oráculo.
- **Consumo de Memória:** Moderado ($O(P \cdot n)$). O consumo é diretamente proporcional ao tamanho da população (P) definida para a evolução. A alocação foi necessária para manter simultaneamente os cromossomos da geração atual e da próxima geração durante as operações de seleção e *crossover*.

3 Análise Teórica: Demonstração Formal da Redução de Complexidade

3.1 Divisão e Conquista: De $O(k^n)$ para $O(2 \cdot k^{n/2})$

A essência do paradigma de Divisão e Conquista neste contexto repousa na fragmentação da senha original, de comprimento n , em duas metades independentes de tamanho $n/2$. Em vez de buscar a solução no espaço global, resolvemos dois subproblemas menores validando partes da *string* isoladamente.

Teorema 1 (Redução de Complexidade por Divisão).

Seja $P = p_1 p_2 \dots p_n$ a senha de n caracteres sobre um alfabeto de tamanho k . Supondo que o Oráculo permita a verificação de substrings em posições arbitrárias, a busca pela senha P pode ser realizada com, no máximo, $2 \cdot k^{n/2}$ chamadas ao sistema validador. **Demonstração.** Dividimos P em dois subproblemas estruturais independentes:

- Subproblema 1: $P_1 = p_1 \dots p_{n/2}$
- Subproblema 2: $P_2 = p_{n/2+1} \dots p_n$

Para descobrir P_1 , executamos uma busca no subespaço de $k^{n/2}$ candidatas possíveis, acionando a validação parcial do Oráculo a cada passo. O número máximo de chamadas para este subproblema é exatamente $k^{n/2}$. Analogamente, para desvendar P_2 , testamos cada candidata na segunda metade da *string*, somando no máximo $k^{n/2}$ consultas adicionais.

Como os dois subproblemas são estritamente desacoplados (o acerto ou erro em P_1 não interfere no processamento de P_2), o custo total de chamadas ao Oráculo (H_{DC}) no pior caso é a simples soma das partes:

$$H_{DC} \leq k^{n/2} + k^{n/2} = 2 \cdot k^{n/2} \quad (1)$$

Corolário (Fator de Redução). A relação de eficiência teórica entre a complexidade da Força Bruta (H_{FB}) e a da Divisão e Conquista (H_{DC}) explicita o colossal ganho computacional desta abordagem:

$$\frac{H_{FB}}{H_{DC}} = \frac{k^n}{2 \cdot k^{n/2}} = \frac{k^{n/2}}{2} \quad (2)$$

Aplicando os parâmetros exatos do nosso experimento ($k = 62$ caracteres possíveis e $n = 8$ posições):

$$\frac{62^8}{2 \cdot 62^4} = \frac{62^4}{2} = \frac{14.776.336}{2} = 7.388.168 \quad (3)$$

Isto comprova matematicamente que, ao fragmentar a senha ao meio, o paradigma de Divisão e Conquista torna-se, no pior caso, aproximadamente **7,38 milhões de vezes mais eficiente** do que a Força Bruta. O modelo reduz o esforço computacional de um limite impraticável de cerca de 218 trilhões de combinações para um domínio altamente viável de apenas 29,5 milhões de consultas.

4 Discussão e Análise Prática

Nesta seção, avalia-se a viabilidade tática dos paradigmas frente a cenários e defesas do mundo real.

4.1 Viabilidade sob Restrição de Acesso (*Rate Limiting*)

Questão: *Com bloqueio de IP após 5 tentativas erradas, qual estratégia seria aplicável?*

Sob esta restrição, métodos de requisições massivas (Força Bruta, Divisão e Conquista e Genéticos) são bloqueados instantaneamente. Apenas a lógica da **Programação Dinâmica** sobrevive em ataques *online*, desde que o atacante utilize a memória (um dicionário filtrado) para testar as 5 senhas estatisticamente mais prováveis. Sem isso, o ataque exige rotação contínua de IPs (uso de *botnets*) ou deve ser executado de forma *offline* sobre dados vazados.

4.2 O Custo do Oráculo na Vida Real

Questão: *O que representaria o "Fitness Score" dos Algoritmos Genéticos na vida real?*

Servidores reais não indicam a quantidade de caracteres corretos. Contudo, os atacantes emulam esse *Fitness Score* por meio de **Ataques de Canal Lateral**, como o Ataque de Tempo (*Timing Attack*). Se a validação do servidor interrompe a verificação no primeiro caractere errado, uma tentativa com o prefixo correto demorará microssegundos a mais para ser rejeitada. Em dispositivos IoT, oscilações no consumo de energia também servem como indicativo prático (*score*) de acerto parcial.

4.3 *Trade-off*: Espaço vs. Tempo

Questão: *Quanto de memória foi necessário para tornar a Programação Dinâmica mais rápida? Valeu a pena?*

No experimento, o custo espacial foi irrisório (dezenas de *bytes* em cache), mas derrubou o tempo de 0,19 s para 0,000022 s e as consultas de 14 milhões para 18. No mundo real, ataques avançados exigem *Gigabytes* ou *Terabytes* de memória (*Rainbow Tables* e dicionários massivos) para pré-computar combinações. Esse investimento vale imensamente a pena, pois o custo financeiro de armazenamento é ínfimo se comparado ao custo de processamento (CPU/GPU) e energia exigidos pela busca exaustiva.

5 Conclusão

Este trabalho evidenciou, teórica e empiricamente, o impacto da escolha do paradigma algorítmico na resolução de problemas de alta complexidade computacional, como a quebra de senhas. Os resultados demonstram de forma inequívoca a inviabilidade da Força Bruta pura frente a grandes espaços de busca. Em contrapartida, abordagens como Divisão e Conquista e Programação Dinâmica provaram ser capazes de colapsar a complexidade temporal de $O(k^n)$ para limites ínfimos e tratáveis, validando na prática o poder matemático do particionamento e da memoização. Adicionalmente, os Algoritmos Genéticos destacaram-se como uma heurística computacionalmente leve, embora sua viabilidade no mundo real dependa da execução de ataques de canal lateral para a medição da aptidão.

A análise prática voltada ao cenário de *Pentesting* reforça que otimizações algorítmicas, embora extremamente eficientes em simulações locais, esbarram nas defesas ativas de infraestruturas *online* modernas, como o *Rate Limiting*. Conclui-se, portanto, que a criptoanálise contemporânea fundamenta-se fortemente no *trade-off* entre espaço e tempo executado de maneira *offline*, onde o investimento em memória compensa largamente o custo do processamento exaustivo.

Em suma, o estudo reitera que a segurança em sistemas de autenticação não deve depender exclusivamente da extensão ou complexidade matemática da senha, mas exige uma arquitetura de validação robusta, rigorosa no controle de acesso e resiliente contra inferências de tempo.

Referências

- [1] CORMEN, T. H. et al. *Introduction to Algorithms*. 4^a ed. MIT Press, 2022.
- [2] SKIENA, S. S. *The Algorithm Design Manual*. 3^a ed. Springer, 2020.
- [3] GOLDBERG, D. E. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.
- [4] ANDERSON, R. *Security Engineering*. 3^a ed. Wiley, 2020. Capítulo 3: Password Security.